

Text Line Processing for High-Confidence Skew Detection in Image Documents

Daniel Rosner, Costin-Anton Boiangiu, Alexandru Stefanescu, Nicolae Tapus, Alexandra Olteanu
Computer Science Department, Faculty of Automatic Control and Computers
University Politehnica of Bucharest
Splaiul Independentei 313, Sector 6, 060042 Bucharest, Romania
Emails: {daniel.rosner, costin.boiangiu, nicolae.tabus}@cs.pub.ro,
{alexandru.stefanescu1708, alexandra.olteanu}@cti.pub.ro

February 1, 2026

Abstract

Skew detection and correction is an important step in automated content conversion systems, on which overall system performance is dependent. Although there are many working solutions at the present time, the search for an algorithm that can achieve good error rates in a fast running time and on different layout types is still open, so new solutions for skew detection are needed. The paper at hand presents a neighbor clustering based approach that has the classical advantages of this class of algorithms — the speed, but delivers better accuracy, comparable with that of Hough based solutions.

Keywords: skew detection; deskew; Hough transform; connected component; neighbor clustering

1 INTRODUCTION

The global shift towards the digitalization of both libraries as well as company archives has pressed for faster and higher accuracy automated content conversion systems.

These automated content conversion systems have a very complex task to handle: making the transition from plain images, that can often be deteriorated, to digital information. A complex, multiple stage architecture is thus required: scanning, image compression and representation, image preprocessing for noise and color correction, binarization, skew detection and correction, page segmentation and character characteristics extraction, character recognition, language detection. Finally, more advanced systems incorporate stages for dictionary based correction and feedback.

The first stage of this complex system represents the physical scanning, and often introduces the most errors, including noise, non-uniform colors, faded background, skew and problems.

There are many reasons for skew faults, but they can be divided in two major categories — errors in the scanning procedure and errors due to scanning images that are already skewed to begin with.

Scanning procedure errors are generally due to paper misalignment of the document to be scanned on the scanning sensor. Problems also arise from the impossibility of placing in a straight line on the scanner certain types of documents — like thick books or large documents.

Documents that are already skewed are most common in company archives, where large quantities of documents are copies that were not properly aligned in the copying machines/fax machines.

The presence of skew defects has a profound negative impact on the OCR system results. Thus, a good skew correction system is vital for this complex system to work properly.

2 RELATED WORK

2.1 Major groups of algorithms

In the past years, four major groups of algorithms have differentiated themselves through many implementations and improvements: projection profile, Hough transforms, clustering of nearest neighbors and

cross correlation. Other methods have been proposed based on hybrid approaches [?], Fourier analysis [?], wavelet decomposition [?], moment [?], morphology [?].

Algorithms based on Hough transform [?] and on projection profiling [?] assume that lines of text are horizontal and parallel in a skew free document, and focus on detecting the angle of the dominant series of parallel lines of pixels, regardless whether this pixels are connected or not. Cross correlation techniques [?] rely on coherency between lines of text while neighbor clustering approaches are based on character grouping and line extraction.

2.2 Nearest neighbor

Nearest neighbor clustering techniques [?], use a bottom-up process to select skew-relevant information, generally by grouping connected components from the original input image. Then, the spatial relationships between the connected components are used to estimate the skew angle of the document. In [?] K-nearest-neighbor chain clustering is used as an improvement to the original method, and is shown to provide better clustering for a more accurate skew estimation.

Generally, nearest neighbor based methods have advantages in terms of speed and a greater tolerance to non-text areas but the disadvantage of low accuracy results. There are many reasons for this, but generally they are the results either of poor clustering or of the influence of descendents. Some nearest neighbor based methods use measurements between pairs of entities, and thus, ascender - descender pairs can introduce errors.

2.3 Presumptions

The general presumptions adopted for most skew detection algorithms are also valid for the proposed algorithm. First, the input images are binary (black and white). The input images are mostly composed of parallel straight lines of text, arranged in blocks, generally in parallel columns.

Expected skew errors are small, since during a normal scan operation, errors above 15° are unlikely to happen.

3 PROPOSED ALGORITHM

The present work was started from the observation that generally, good skew features can be extracted from the borders of image entities. In the case of characters, especially Latin characters, bottom border-lines can provide excellent measurement of skew angles, since in most languages that use Latin characters in writing, ascenders characters are more common than descenders in a normal text.

Thus, the goal of this algorithm is to identify lines of text and extract skew angle information based on the bottom line formed by them. In order to improve accuracy, several improvements were introduced.

3.1 Preprocessing

The algorithm is designed and implemented to work on binary images. In the following we will refer to entities as all the black connected components resulting from the document image binarization.

Connected components are extracted from the binary image to serve as input for the algorithm itself. A list of entities is obtained, and only the 4 corners coordinates are preserved.

3.2 Entities filter

In order to eliminate noise, two separate thresholds are used, both based on the minimal size at which one can consider an entity to be a character - size mainly dictated by the minimum size at which a human could possibly identify a characters features.

The first threshold is static - 6 pixel height. The point of reference for measuring resolution is the lower case letter 'e' (six points letter). The reason is that this is the smallest letter used in newspapers (e.g. in the stock market reports). When this letter is accurately reproduced, all six point letters, as well as larger fonts, will be easily legible.

The second threshold is dynamic, and it depends on the image resolution. As the human eye can distinguish, unaided, details of as small as 0.1mm no entity below 0.6mm of height should be considered as it is the minimum required for accurately representing the letter 'e'. For example, in the case of a 300

DPI scan, any entity below 7 pixels in height will be discarded. Thus, the minimum height, in pixels, will be:

$$\text{Min}_{\text{height}} = \text{round} \left(\max \left[6, \frac{0.6 \cdot \text{imageDPI}}{25.4} \right] \right). \quad (1)$$

For eliminating large pictures, we use another two thresholds. One is static - we eliminate all entities higher than 20% of the input image size. The other one comes from a fast statistic on the remaining entities (after removing noise from the image) - entities larger than 250% the average entity height are discarded. Thus, the maximum height, measured in pixels, will be:

$$\text{Max}_{\text{height}} = \text{round} \left(\min \left[\frac{\text{img}_h}{5}, 2.5 * \text{avg}_h \right] \right). \quad (2)$$

Although this can lead to discarding larger characters, this will eliminate at most a few lines of text and will have little impact on the global accuracy of the algorithm.

For special cases of images made up of only a few sparse words that might not comply with the maximum height criteria and provide no input data, a second pass in which all entities are considered can be made.

3.3 Clustering

In order to obtain accurate results, skew angle is measured along lines of text. To detect these lines of text, we developed a simple clustering technique that will produce accurate results on skewed documents in the area of $\pm 20^\circ$.

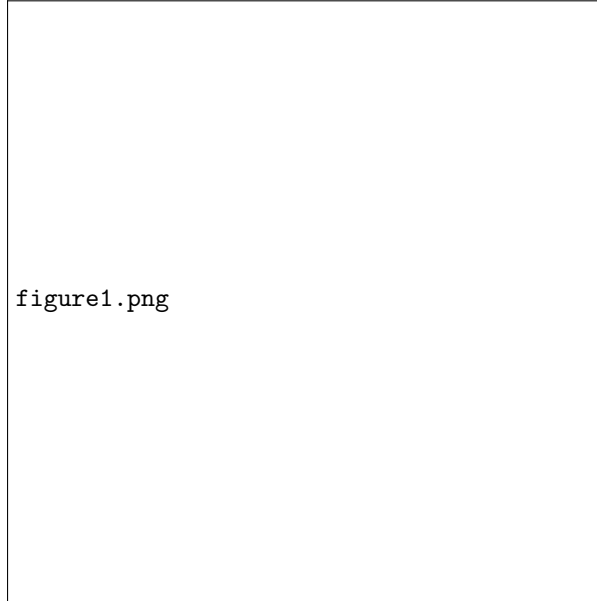


Figure 1: Clustering neighborhood

The clustering algorithm is based on rectangular neighborhood covering. That is, for every character, we construct a neighborhood of equal height and of length of twice the character's height, starting from its right bottom pixel. Another block will be placed in this neighborhood, if its middle-y-line intersects this neighborhood.

3.4 Cluster selection

The proposed clustering algorithm will produce a series of clusters, some of them representing accurately-detected lines of text, some of them representing lines formed from small images, lines from misaligned columns, and other types of errors. It is thus necessary to establish a threshold at which to accept clusters as being good enough for skew estimation. Generally, longer clusters contain better skew information.

There is a direct link between the accuracy we are searching for and the length, in pixels, of the cluster. In order to achieve a certain angular precision, further denoted as θ , a minimum amount of

information is needed. If measurements are taken along a line of cluster len pixels, then the accuracy that can be achieved depends on the angle described by the triangle between this line and 1 pixel height (because below one pixel no more refined information can be obtained).

Thus, the minimum size, in pixels, for a cluster in order to achieve a certain angular precision is:

$$\text{Min}_{\text{cluster_len}} = \frac{1}{\tan \theta}. \quad (3)$$

However, there are images containing multiple columns that might not satisfy this condition. For this purpose, a series of tests have been conducted and a second threshold calculus that can be used for such scenarios has been developed:

After running tests on over 60 images at different skew angles, with different values for α and β the value pair (10, 10) has been determined to be optimal for best overall results.

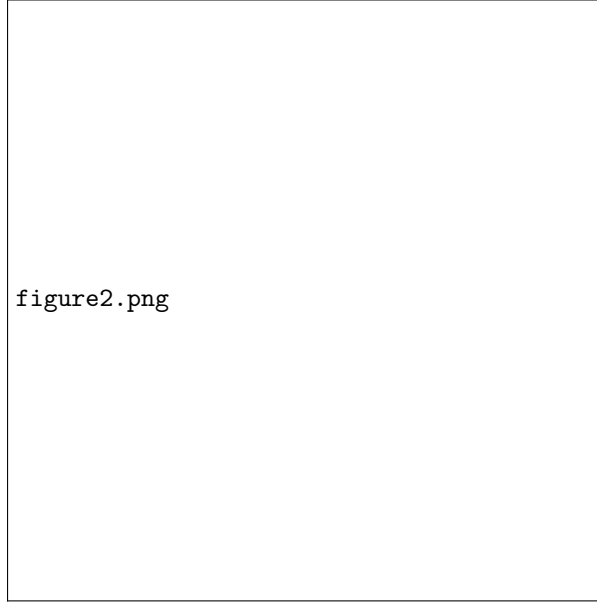


Figure 2: Sample from image in Fig. 3, showing clusters with skew lines

3.5 Skew estimation for each cluster

For each selected cluster the skew angle is computed. As discussed above, the bottom pixels of characters are ideal candidates for skew detection. Thus, a least-square line-fitting method is used to determine the line that fits best through the bottom pixels of the entities in the clusters.

Considering the coordinates (x, y) of the middle bottom points of the entities in the cluster, the slope will be:

$$\theta = \text{atan} \left(\frac{n \sum xy - (\sum x)(\sum y)}{n \sum x^2 - (\sum x)^2} \right). \quad (5)$$

As descenders in the text line will introduce errors, it is necessary to rule them out of the computation. First, an initial line-fitting is computed in order to determine the direction of the text line (positive angle or negative angle). After this, a second line-fitting computing step is performed, again with the least-square method, during which letters that have their bottom pixels out of order are not considered (that is, if the line is ascending, for example, characters that have their bottom pixel below their predecessors' bottom pixels will be ignored). This removes most of the descenders from the computation and improves the skew angle detection precision.

3.6 Global skew angle

As the present algorithm is meant to determine skew angle within images with a single predominant skew, it uses a heuristic based on a weighted average value, where every angle's weight is given by its generating cluster length.

In case of lack of clusters meeting the requirements, two options can be considered: returning an error, or computing the angle using more permissive thresholds, but throwing a warning regarding the low accuracy of the results. Tests showed that in images where sufficient long clusters were not present, the use of the available shorter clusters lead to an increase in skew angle detection error of up to 1° .

4 RESULTS

The algorithm was run on a database of images selected for skew detection, performing over 400 tests. Images were chosen so as to test input at different DPI (150, 300), various size (from 800×1000 to 5000×7000 pixels) and with various layouts.

Some images were obtained by rotating 30 unskewed PDF files at 8 different angles, for a total of 240 tests, with results shown in Table I.

Another 28 scanned images for which the ground truth (the real 0° position) was determined manually were used in testing. The results of the 196 tests performed are shown in Table II.

The error rates for the second set of documents are slightly higher, as the manual determined ground truth is not 100% accurate. However, the tests motivated the use of a Hough transform based implementation as a benchmark for skew detection accuracy. The maximum error obtained was 0.1° for the images with real 0 degree ground truth, and 0.25° for images with manually detected ground truth.

The average time for the proposed algorithm is not skew angle dependent. Unlike Hough or Projection Profiling techniques, there is no desired resolution setting, and thus the running time cannot be modified according to the needed precision.

The tests were carried out on a 2 GHz Intel Core Duo processor. For an image at 5000×7000 pixels, the average running time was below 1 second.

Table 1: Average error for Hough & new algorithm on image set 1

Skew Angle	0.4°	0.7°	1°	3°	1.32°	4.4°	5°	8.6°
Hough Average Error	0.02°	0.02°	0.03°	0.03°	0.03°	0.05°	0.04°	0.04°
New Algorithm Average Error	0.07°	0.07°	0.03°	0.04°	0.03°	0.05°	0.04°	0.07°

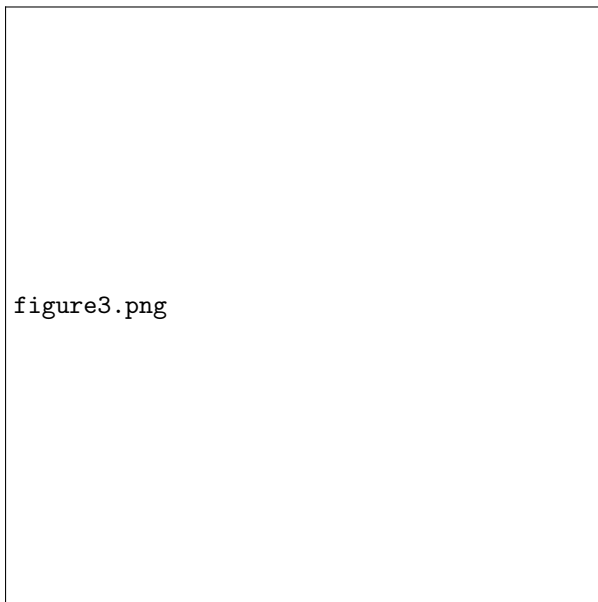


Figure 3: Large image with predominant picture content

The running time is also dependent on the amount of text in the input image. For images containing little text, the running time is decreased drastically up to 10X, with little loss of accuracy. For example, Fig. 3 has a size of 3936×5108 pixels and a skew angle of 5 degrees. The skew angle was determined as 5.028° .

Table 2: Average error for Hough & new algorithm on image set 2

Skew Angle	1°	2°	3°	3.7°	5°	7.5°	9.5°
Hough Average Error	0.12°	0.06°	0.06°	0.08°	0.07°	0.07°	0.09°
New Algorithm Average Error	0.06°	0.06°	0.09°	0.06°	0.09°	0.09°	0.10°

Fig. 4 comprises of several narrow columns, with different font sizes, different alignments and numerous black separators, thus forming an unfavorable input image. The skew angle is 5° and is detected as 5.181°.

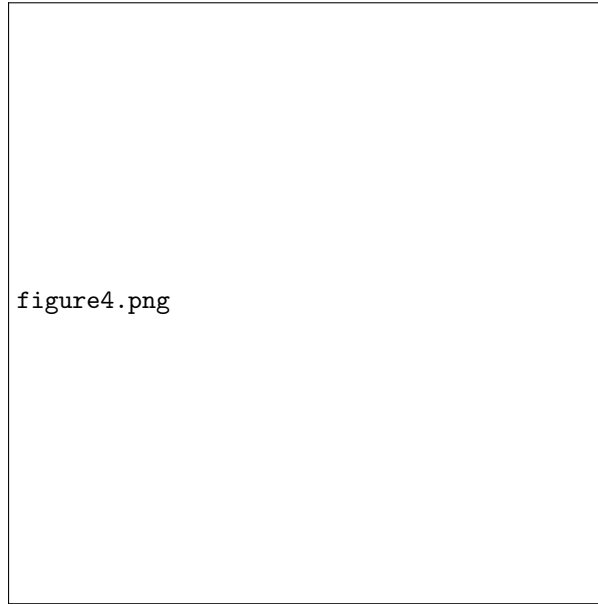


Figure 4: Image with narrow, misaligned columns, with different font sizes

5 CONCLUSIONS

5.1 Advantages

Compared with other algorithms from its class, it obtains better results in terms of error rates. Error rates decrease well below 0.1° with the increase in the size of the input document.

Due to the multi-stage nature and the dynamically-computed thresholds used in the proposed algorithm, it is less sensitive to errors than other single-stage algorithms based on textual line information. This means that for any required skew detection precision, less textual information is actually required. Again, this means that despite the fact that, theoretically, the proposed algorithm introduces just some new processing phases, from a practical point of view the detected skew angle will be more precise than in other approaches based on the same ideas because it requires less textual information to achieve the same skew precision.

One final advantage is that during the skew computation algorithm, thresholds necessary to achieve certain accuracy are computed. This means that the algorithm can assert if the input data does not provide sufficient information for an accurate skew estimation, and can return a warning that includes the approximate expected error, given the characteristics of the input document.

5.2 Disadvantages

The current implementation is slower than other neighbor based approaches, but fast enough for an OCR solution.

As the algorithm is partially based on heuristic findings, it cannot be mathematically proven to be reliable. However, the selection of necessary thresholds for achieving certain accuracy was shown to be

scientifically founded, and the algorithm can assert when the required accuracy cannot be obtained.

5.3 Future Improvements

As the algorithm can run with good results on input with only small quantities of text, the next goal is to develop a fast algorithm for choosing a textual area from an image. Based on the same observation, another possible expansion would be towards a multi-skew-angle detection solution that could be used to detect blocks of text skewed at different angles within the same document.

ACKNOWLEDGMENT

The research presented in this paper is supported by the national project "Excelentia in cercetare prin programe postdoctorale in domenii prioritare ale societatii bazate pe cunoastere (EXCEL)", Project POSDRU/89/1.5/S/62557.